

Generative AI and Large Language Models

BMED365: Topic List G01–G20

BMED365

Spring 2026

Overview

- 1 Introduction to Generative AI
- 2 Transformer Architecture
- 3 LLM Fundamentals
- 4 Prompt Engineering
- 5 Context Engineering
- 6 Challenges with LLM
- 7 Models and Advanced Concepts

G01: Define generative AI og skille fra diskriminativ AI

Diskriminativ AI:

- Lærer å **klassifisere** or skille mellom kategorier
- Modorer $P(y|x)$ – sannsynlighet for klasse y gitt input x
- Examples: “Er dette kreft?”, “Hvilken sykdom?”
- CNN for bildeclassification

Generative AI:

- Lærer å **skape** nytt innhold
- Modorer $P(x)$ or $P(x|y)$ – datafordelingen selv
- Examples: Tekst, bilder, kode, musikk
- LLM, diffusjonsmodor

Key difference

Diskriminativ: Input $x \rightarrow$ Kategori/Label y | **Generativ:** Prompt $\rightarrow$ Nytt innhold

Medical application

Generative AI: Oppsummere journaler, generere rapporter, svare på spørsmål, syntetisere trainingsdata

G02: Explain self-attention-mekanismen på et konseptuelt nivå

Self-attention: “What is relevant for hva?”

Intuisjon:

- For hvert ord: Se på **alle** andre ord i setningen
- Beregn en **vektet sum** basert på relevans
- Fanger opp avhengigheter over lange avstander

Example:

*“Pasienten tok **medisinen**, og **den** hjalp mot smertene.”*

- Self-attention kobler “den” til “medisinen” (ikke “smertene”)

Query, Key, Value (QKV)

Hvert token genererer tre vektorer:

Query “Hva leter jeg etter?” | **Key** “Hva kan jeg tilby?” | **Value** “What is innholdet mitt?”

G03: Describe transformer-arkitekturer (encoder-decoder)

Transformer Architecture (Vaswani et al., 2017):

Encoder:

- Prosesserer input-sekvens
- Bygger kontekstrik representasjon
- Brukes i: BERT, embeddings

Decoder:

- Genererer output sekvensielt
- Bruker maskert self-attention
- Brukes i: GPT, Claude, Gemini

Full encoder-decoder:

- Oversettelse, oppsummering
- T5, Flan-T5

Nøkkelkomponenter:

- 1 Multi-head self-attention
- 2 Feed-forward nettverk
- 3 Layer normalization
- 4 Residual connections
- 5 Positional encoding

GPT = "Decoder-only"

Store language models (LLM) bruker typisk kun decoder-delen for tekstgenerering.

G04: Explain hva tokens er og hvordan tokenisering fungerer

Tokens = tekstens “atomer”

- LLM-er leser ikke bokstaver or ord – de leser **tokens**
- Token $\approx$ delord, ca. 4 tegn per token (for engelsk/norsk)

Tokenisering – eksempel:

“Pasienten har diabetes” $\rightarrow$ [“Pas”, “ient”, “en”, “ har”, “ diab”, “etes”]

BPE og språkforskjor:

- Bygger vokabular iterativt
- GPT-4: ca. 100 000 tokens
- **Kinesisk:** Hvert tegn $\approx$ 1–2 tokens
- Forenklet/tradisjonell: Ulike tokens

Important å vite:

- Lange/sjeldne ord = flere tokens
- Påvirker kostnad og hastighet
- **tiktoken** (OpenAI) for telling
- Ulike modor = ulike tokenizers

Why tokens?

Mer effektivt enn bokstaver (for kort) or hele ord (vokabular for stort).

G05: Understand konseptet kontekstvindu (context window)

Kontekstvindu = modellens “arbeidsminne”

- Maksimalt antall tokens modellen kan “se” samtidig
- Inkluderer både **input** (prompt) og **output** (svar)

Examples på kontekstvinduer:

Modell	Kontekst	≈ Sider
GPT-4o	128K	~150 sider
GPT-5	256K+	~300 sider
Claude Opus 4.5	200K	~250 sider
Gemini 3 Pro	2M+	Hele bøker!

Input vs. output:

- **Input-kontekst:** Hvor mye modellen kan lese
- **Output-kontekst:** Hvor langt svar den kan gi
- Output ofte mindre (4K–32K tokens)

Limitations

For lange tekster må kuttes. Modellen “glemmer” utenfor vinduet. Større vindu = dyrere/tregere.

G06: Explain hva temperatur betyr i tekstgenerering

Temperatur = kontroll over “kreativitet”

Teknisk:

- Skalerer logits før softmax: $p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$
- **Effekt:** Lav $T \rightarrow$ skarpere fordeling (høyeste logit dominerer); Høy $T \rightarrow$ flatere fordeling (mer tilfeldighet)

Lav temperatur (0.0–0.3):

- Mer deterministisk
- Velger mest sannsynlige tokens
- Konsistent, “trygt”
- ✓ Medical info, fakta

Høy temperatur (0.7–1.0+):

- Mer tilfeldig/kreativ
- Flere overraskende valg
- Mer variasjon
- ✓ Kreativ skrijving, brainstorming

Recommendation for medicine

Bruk **lav temperatur** (0.0–0.3) for faktabaserte oppgaver. Høyere temperatur øker risiko for hallusinerer.

G07: Apply zero-shot prompting

Zero-shot = ingen eksempler – be modellen løse oppgaven **uten å vise eksempler**

Examples på zero-shot prompts:

Zero-shot prompts

- 1 Klassifiser symptom som akutt/kronisk: “Hodepine i 3 mnd”
- 2 Oppsummer journalnotatet i 3 setninger.
- 3 Oversett til pasientvennlig språk: “Bilateral pneumoni”
- 4 Ekstraher alle medikamenter fra denne teksten.
- 5 Er denne lab-verdien unormal? Hb 8.2 g/dL

Når fungerer zero-shot?

- Modellen har sett lignende oppgaver
- Oppgaven er tydelig definert

Limitations

For spesialiserte oppgaver kan zero-shot gi dårlige resultater. Da trengs **few-shot**.

G08: Apply few-shot prompting

Few-shot = noen eksempler

- Gi modellen **2–5 eksempler** på ønsket input-output
- Modellen lærer mønsteret “in-context”

Few-shot prompt

Klassifiser symptomvarighet:

Symptom: ‘Brystsmerter i 30 minutter’ → Akutt

Symptom: ‘Ryggsmerter i 2 år’ → Kronisk

Symptom: ‘Feber siden i går’ → Akutt

--

Symptom: ‘Hodepine i 3 måneder’ →

Tip for effektiv few-shot:

- Velg **representative** eksempler
- Vis **variasjonen** i mulige outputs
- Hold eksemplene **konsistente** i format
- Start med 3–5 eksempler, juster ved behov

G09: Apply Chain-of-Thought (CoT) prompting

Chain-of-Thought = vis resonnementet

- Be modellen **tenke steg-for-steg**
- Forbedrer ytelse på komplekse resonneringsoppgaver

Uten CoT

Bør pasienten henvises?
→ “Ja”

Med CoT

Tenk steg-for-steg:
1. Symptomer: X, Y, Z
2. Alvorlighetsgrad: Moderat
→ Konklusjon: Bør henvises

Varianter:

- **Zero-shot CoT:** “Let’s think step by step”
- **Few-shot CoT:** Vis eksempler med resonnement

Built-in CoT i nyere LLM-er

ChatGPT o1/o3 (Thinking), Claude Opus 4.5 (Extended Thinking), Gemini 3 (Deep Think) har CoT innebygd.

G10: Describe god praksis for systemprompts

Systemprompt = instruksjon som setter kontekst

Elementer i et godt systemprompt:

- 1 **Rolle:** "Du er en medisinsk assistent..."
- 2 **Kontekst:** "Brukeren er helsepersonell..."
- 3 **Oppførsel:** "Svar konsist og presist..."
- 4 **Limitations:** "Ikke gi diagnoser..."
- 5 **Format:** "Svar på norsk, bruk punktlister..."

Example på systemprompt

Du er en medisinsk AI-assistent for helsepersonell. Svar på norsk. Vær presis og evidensbasert. Henvis til kilder når mulig. Gi aldri definitive diagnoser - foreslå differensialdiagnoser. Ved usikkerhet, si fra.

Tip

Test og iterer på systemprompts! Små endringer kan gi stor effekt.

G15: Fra Prompt Engineering til Context Engineering

Prompt Engineering vs. Context Engineering:

- **Prompt engineering:** Fokus på å utforme gode spørsmål
- **Context engineering:** Designe *hele kontekstvinduet* strategisk

Komponenter i context engineering:

Komponent	Descrripsjon	Medical example
System prompt	Rolle og instruksjoner	"Clinical beslutningsstøtte"
Bruker-prompt	Spesifikt spørsmål	"Vurder pasientens symptomer"
Verktøy	Tilgjengelige handlinger	Lab-søk, ICD-10, Felleskatalogen
RAG-kontekst	Hentet informasjon	Retningslinjer, journal, artikler
Samtalehistorikk	Tidligere dialog	Tidligere spørsmål og svar
Strukturerte data	Formattert info	JSON med vitale tegn

Why viktig i medisin?

Rik kontekst → sikrere, mer relevante og personaliserte svar

G16–G17: Modellspesifikke prompting-teknikker

Tre ledende LLM-leverandører med ulike styrker:

GPT-5.2 (OpenAI):

- Sterkest på strukturert output
- Beste verktøyintegrasjon
- God instruksjonsetterlevelse

Godt egnet for:

- JSON-ekstraksjon
- API-integrasjoner
- Agentiske systemer

Claude Opus 4.5:

- 200K tokens kontekst
- Nyansert resonnering
- Constitutional AI

Godt egnet for:

- Ethiske dilemmaer
- Lang journalanalyse
- Pasientkommunikasjon

Gemini 3 (Google):

- Multimodal (bilde+tekst)
- Sanntids websøk
- Vitenskapelig styrke

Godt egnet for:

- Image Analysis
- Litteratursøk
- Beregninger

Lab 3 Notebook 04

Inneholder detaljerte prompting-guider for hver modell + sammenligningseksperimenter

G18: Håndtere usikkerhet og hallusinasjonsrisiko

Prompt-mønster for medisinsk usikkerhetshåndtering:

Anti-hallucination prompt (GPT-5.2 stil)

<usikkerhet_og_tvetydighet>

- Ved tvetydige symptomer: Presenter differensialdiagnoser med sannsynlighet
- Oppgi tydelig kunnskapens begrensninger og dato for oppdatering
- Henvis alltid til gjeldende retningslinjer og klinisk vurdering
- Ved usikkerhet: Eshellér til høyere hastegrad

</usikkerhet_og_tvetydighet>

Selvsjekk for høyrisiko-oppgaver:

- 1 Skann eget svar for usagte antakelser
- 2 Verifiser at tall og påstander er forankret i kontekst
- 3 Unngå for sterk språkbruk ("alltid", "garantert")
- 4 Kvalifiser konklusjoner med usikkerhetsgrad

Rule of thumb

AI = beslutningsstøtte, **ikke** beslutningstaker. Alltid klinisk validation!

G19–G20: Strukturert ekstraksjon og modellsammenligning

G19: Strukturert ekstraksjon:

JSON-skjema for epikrise

```
{
  diagnose": {
    tekst": string,
    "icd10": string | null
  },
  legemidler": [{
    navn": string,
    dose": string
  }]
}
```

Importante regler:

- Bruk `null` for manglende felt
- Krev kildereferanser
- Verifiser mot originaldokument

G20: Beslutningsmatrise:

Oppgave	Modell
Strukturert output	GPT-5.2
Etiske vurderinger	Claude
Image Analysis	Gemini
Lang kontekst	Claude
Sanntidssøk	Gemini
Verktøy/API	GPT-5.2

Praktisk tips:

- 1 Test samme prompt på 2–3 modor
- 2 Sammenlign mot evaluationskriterier
- 3 Velg modell basert på oppgave
- 4 Dokumenter valg for etterprøvbarehet

G11: Define hallusinerer og dens implikasjoner

Hallusinerer = modellen “finner på” feil informasjon

- LLM-er genererer tekst som *høres* riktig ut, men er *faktisk feil*
- Også kalt *konfabulering* – begge termer brukes i litteraturen

Types of Hallucinations:

- **Faktiske feil:** Feil statistikk, datoer, navn
- **Oppdiktete kilder:** Referanser som ikke eksisterer
- **Logiske feil:** Inkonsistente resonnementer

Utvikling:

- Nyere modor hallusinerer *mindre*
- Kan *ikke* elimineres helt
- RAG og CoT reduserer risiko

Implikasjoner i medisin

Feil dosering kan være livstruende. **Alltid verifiser** mot pålitelige kilder! LLM = beslutningsstøtte, **ikke** erstatning for fagkunnskap.

G12: Know about GPT-5, Claude, Gemini og deres applylser

Modell	Utvikler	Styrker	Application
GPT-5/o3	OpenAI	Multimodal, reasoning	ChatGPT, Copilot
Claude 4	Anthropic	200K kontekst, "trygg"	Dokumentanalyse
Gemini 3	Google	Integrert i Colab	Kodehjelp, søk
Llama 4	Meta	Open source, lokalt	Forskning

Lokale modor:

- Ollama, LM Studio
- Destillering: Liten modell lærer fra stor
- Kvantisering: Redusert precision (16→4 bit)
- GDPR-vennlig, offline

chat.uib.no

Styrker: GDPR-ok, ingen datalagring, gratis for UiB

Svakheter: Eldre modell, begrenset kontekst, ikke multimodal

G13: Explain konseptet grunnmodell (foundation model)

Foundation model = pretraint på enorme datamengder

- Trent på terabytes av tekst/bilder uten spesifikk oppgave
- Kan tilpasses (*finetuned*) til mange forskjellige oppgaver
- Koster millioner å trene fra scratch

Karakteristikk:

- 1 **Stor shalla:** Milliarder av parametre
- 2 **Selvveiledet læring:** Ingen labels nødvendig
- 3 **Emergente egenskaper:** Evner som “dukker opp” ved shalla
- 4 **Tilpasningsbar:** Finetuning, prompting, RAG

Examples på foundation models (2025)

Tekst: GPT-5, Claude 4, Llama 4 | Bilder: DALL-E 3, Stable Diffusion 3 | Multimodal: Gemini 3, Sora

G14: Describe RAG (Retrieval-Augmented Generation)

RAG = koble LLM til ekstern kunnskapsbase

Problemet RAG løser:

- LLM har “frossen” kunnskap
- Kan hallusinere fakta
- Mangler domene-spesifikk info

How RAG fungerer:

- 1 **Spørring:** Bruker stiller spørsmål
- 2 **Retrieval:** Hent dokumenter
- 3 **Augmentation:** Legg til prompt
- 4 **Generation:** LLM svarer

Medical application

- Koble LLM til retningslinjer, [Felleskatalogen](#), [UpToDate](#)
- Svare basert på pasientjournal (med tilganger)
- Reduserer hallusinerer ved å forankre svar i kilder
- Clinical beslutningsstøtte med oppdatert evidens
- Søk i institusjons-spesifikke prosedyrer og protokoller

Summary: Generative AI og LLM

Fundamental:

- G01: Generativ vs. diskriminativ AI
- G02–G03: Self-attention og transformer-arkitektur
- G04–G06: Tokens, kontekstvindu, temperatur

Prompt og context engineering:

- G07–G10: Zero-shot, few-shot, CoT, systemprompts
- G15–G17: Context engineering, modellspesifikke teknikker
- G18–G20: Usikkerhetshåndtering, strukturert ekstraksjon

Utfordringer og avansert:

- G11: Hallusinerer – kritisk i medisinsk kontekst
- G12–G14: Modor (GPT-5.2, Claude, Gemini), RAG

Lab 3 Notebook 04

Detaljerte prompting-guider for GPT-5.2, Claude Opus 4.5 og Gemini 3 med sammenligningseksperimenter.